Automated Product Image Tagging with Deep Learning

Classifying Fashion-MNIST product photos: model comparison and recommendation

Prepared by: [Your Name] | October 2026

Executive Summary

We built and compared three deep learning models that predict a clothing item's product category from its photo, with the goal of automating catalog tagging for an online retailer.

- **Recommendation:** deploy the **Regularized CNN**. On 10,000 held-out test images it reached **94.10% accuracy** (macro F1 0.941).
- **Improvement over the baseline:** compared with a standard fully connected network (88.63% test accuracy), the recommended model makes **48% fewer tagging errors**.
- **Business operating point:** if only predictions with confidence of at least **0.95** are auto-tagged, **79.7% of images** are tagged automatically at **99.25% accuracy**. The remaining 2,032 of 10,000 go to a human reviewer.
- **Main weakness:** upper-body garments look alike at this resolution. **Shirt** is the hardest category (F1 0.81). Most errors are confusions among T-shirt/top, Pullover, Coat and Shirt.

1. Main Objective

The objective is **prediction**: given a product photo, assign the correct category out of ten. Interpretability is secondary, but we still analyze which categories the model confuses, because that determines how the model should be used in practice.

Value to stakeholders. Merchandising teams tag thousands of new product photos by hand, which is slow, costly and inconsistent. An accurate image classifier can tag most new listings instantly and route only uncertain cases to people. This shortens time-to-listing, lowers labor cost and improves search and filtering quality for shoppers.

2. Dataset Description

We used **Fashion-MNIST**, a public dataset of Zalando product photos released as a drop-in benchmark for image classification. It is distributed with the Keras library.

Attribute	Description
Size	70,000 images: 60,000 for training and 10,000 for testing
Image format	28 x 28 pixels, grayscale, item centered on a black background
Features	784 pixel intensities per image (integers 0-255)
Target	Product category, 10 classes: T-shirt/top, Trouser, Pullover, Dress, Coat, Sandal, Shirt, Sneaker, Bag, Ankle boot
Class balance	Perfectly balanced: 6,000 training and 1,000 test images per class
Data quality	No missing values and no exact duplicate images; every image is a complete 28 x 28 array



Figure 1. Random training images for each category.

3. Data Exploration, Cleaning and Feature Engineering

Exploration

- **Balanced classes** mean plain accuracy is a fair headline metric, and no resampling or class weighting is needed.
- **Average images per category** (Figure 2) show that footwear, trousers and bags have distinctive silhouettes, while the upper-body garments share almost the same outline.
- **Similarity of average images** confirms this. The most similar category pairs are Coat / Shirt (cosine 0.987), Pullover / Coat (cosine 0.984), Pullover / Shirt (cosine 0.983). This predicted the main source of model errors before any model was trained.

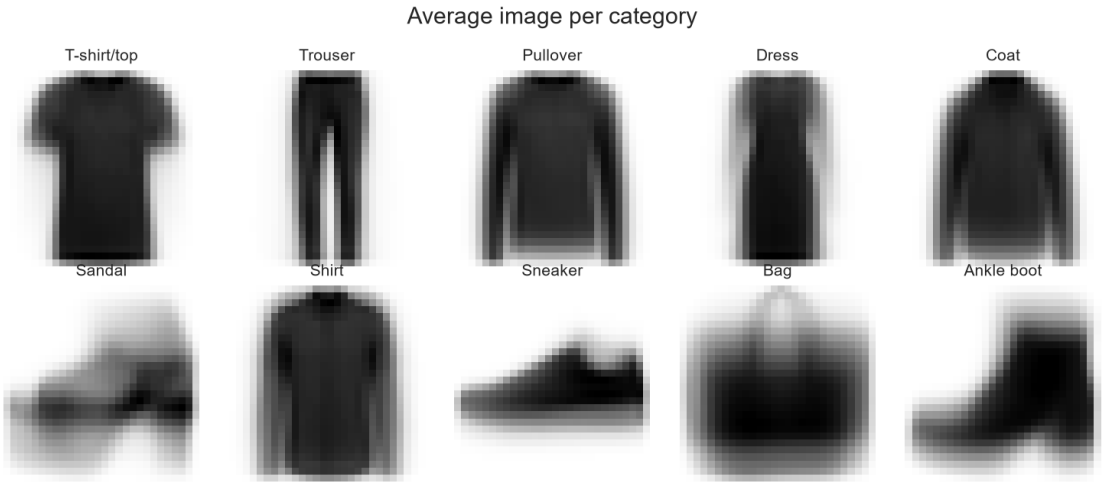


Figure 2. Average image per category. Upper-body garments are nearly indistinguishable by outline.

Cleaning and preprocessing

- **Scaling:** pixel values were divided by 255 to the range [0, 1] for stable gradient-based training.
- **Reshaping:** images were given an explicit channel dimension (1 x 28 x 28, the channels-first layout native to PyTorch), as convolutional layers require.
- **Validation split:** 10,000 training images (stratified by category) were held out to compare models, trigger early stopping and adjust the learning rate.
- **Untouched test set:** the 10,000 official test images were used only once, to evaluate the final selected model.

Feature engineering

Deep learning replaces manual feature engineering: convolutional layers learn their own edge, texture and shape detectors from raw pixels. The one engineered input transformation is **data augmentation** in the third model. Random shifts of up to about 2 pixels during training teach the model that an item's exact position doesn't change its category.

4. Model Training: Three Variations

All three models used the same training procedure, so differences in results come from the architecture: the Adam optimizer (learning rate 0.001), cross-entropy loss, batch size 128, and **early stopping** that restores the weights from the epoch with the lowest validation loss.

- **Model 1, MLP baseline:** a fully connected network (flatten, then dense layers of 256 and 128 units). It treats the image as a flat list of pixels and ignores spatial structure.
- **Model 2, Basic CNN:** two convolution and max-pooling blocks (32 and 64 filters) followed by a 128-unit dense layer. It learns local patterns, such as collars, sleeves and soles, wherever they appear in the image.
- **Model 3, Regularized CNN:** a deeper CNN with two convolutions per block, **batch normalization**, **dropout** (25% after each block, 50% before the output), **data augmentation** (random shifts), and a **learning-rate schedule** that halves the rate when validation loss plateaus.

Model	Parameters	Best / total epochs	Training time	Train accuracy	Validation accuracy	Overfitting gap
MLP baseline	235,146	9 / 14	26 s	91.65%	89.54%	2.11 pts
Basic CNN	421,642	11 / 16	48 s	96.60%	92.69%	3.91 pts
Regularized CNN	871,978	39 / 40	4.8 min	96.54%	94.93%	1.61 pts

Train and validation accuracy were both measured in inference mode after training. The overfitting gap is train minus validation accuracy, in percentage points.

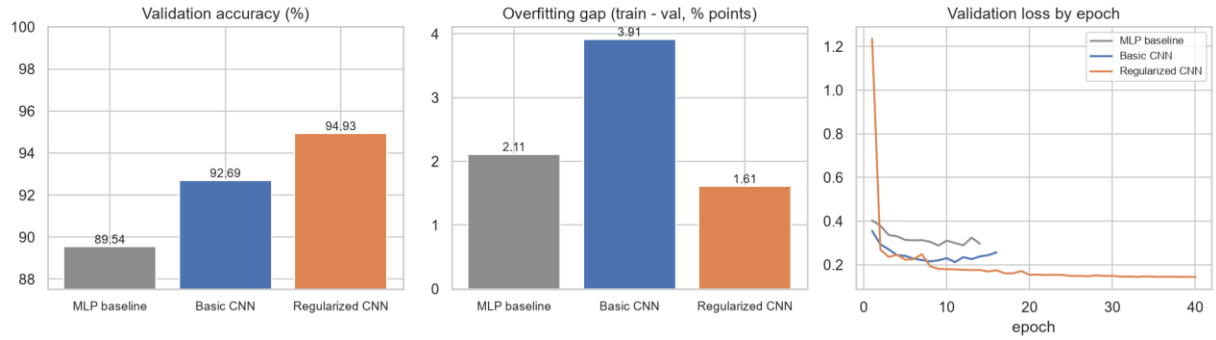


Figure 3. Validation accuracy, overfitting gap and validation loss per epoch for the three models.

What the training curves show

- **The MLP baseline** plateaued at 89.54% validation accuracy. Without spatial awareness, it can't reliably tell apart garments that differ only in local details.
- **The basic CNN** improved validation accuracy to 92.69%, confirming the value of convolution. Its validation loss began rising after epoch 11 while training loss kept falling (Figure 4), a classic sign of **overfitting**: train accuracy reached 96.60%.
- **The regularized CNN** reached 94.93% validation accuracy with an overfitting gap of only 1.61 pts, compared with 3.91 pts for the basic CNN. Training and validation curves stay close together (Figure 5). The training curve sits slightly below validation only because dropout and augmentation are active during training. Its best epoch was 39 of 40, so it was still improving slowly at the epoch limit.

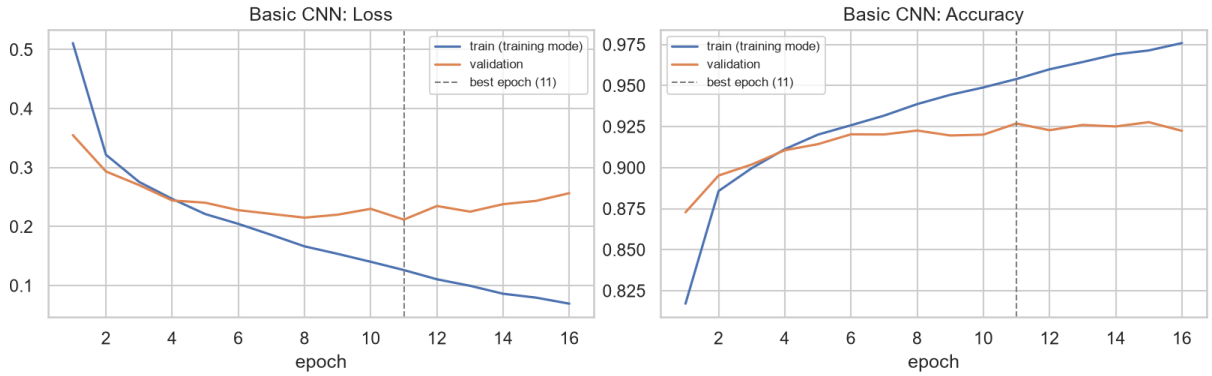


Figure 4. Basic CNN: validation loss starts rising while training loss keeps falling, which signals overfitting.

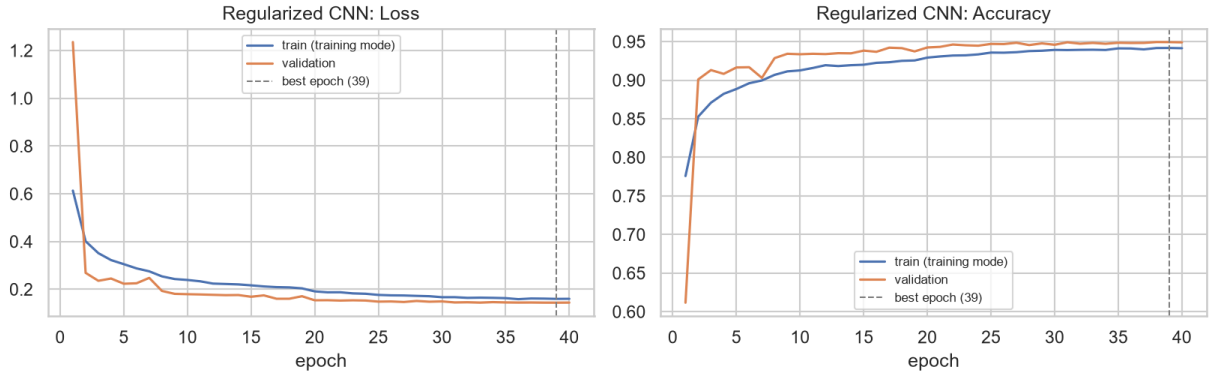


Figure 5. Regularized CNN: augmentation, dropout and batch normalization keep training and validation curves aligned.

5. Recommended Model and Test Results

We recommend the **Regularized CNN** because it had the **highest validation accuracy** and the **smallest overfitting gap**, which makes it the most likely to hold up on new images. Its extra training time (4.8 min on one GPU) is a one-time cost. Predictions take only milliseconds per image.

On the untouched test set, the Regularized CNN reached **94.10% accuracy** and a **macro F1 of 0.941** (590 errors out of 10,000). For transparency, the other models scored 88.63% (MLP baseline) and 91.72% (Basic CNN) on the same test set. This confirms the ranking chosen on validation data.

Category (hardest first)	Precision	Recall	F1-score
Shirt	0.818	0.807	0.813
T-shirt/top	0.901	0.885	0.893
Coat	0.890	0.935	0.912
Pullover	0.930	0.909	0.920
Dress	0.941	0.949	0.945
Sneaker	0.967	0.986	0.976
Ankle boot	0.984	0.969	0.976
Sandal	0.993	0.989	0.991
Trouser	0.994	0.989	0.991
Bag	0.992	0.992	0.992

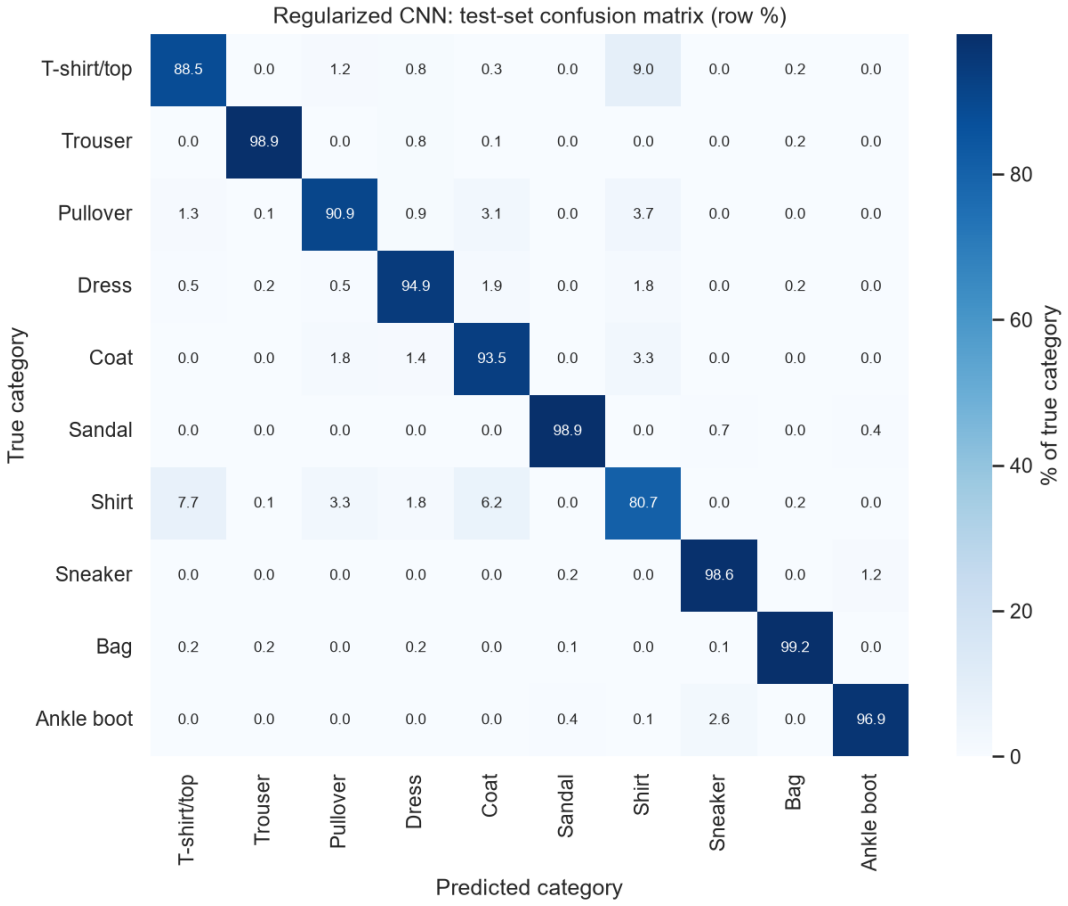


Figure 6. Test-set confusion matrix (each row sums to 100%). Errors concentrate among upper-body garments.

The most frequent errors were T-shirt/top predicted as Shirt (90); Shirt predicted as T-shirt/top (77); Shirt predicted as Coat (62); Pullover predicted as Shirt (37). Among the top 8 error types, **363 of 389** errors were confusions between upper-body garments. Many of these images are ambiguous even to a person at 28 x 28 resolution (Figure 7).

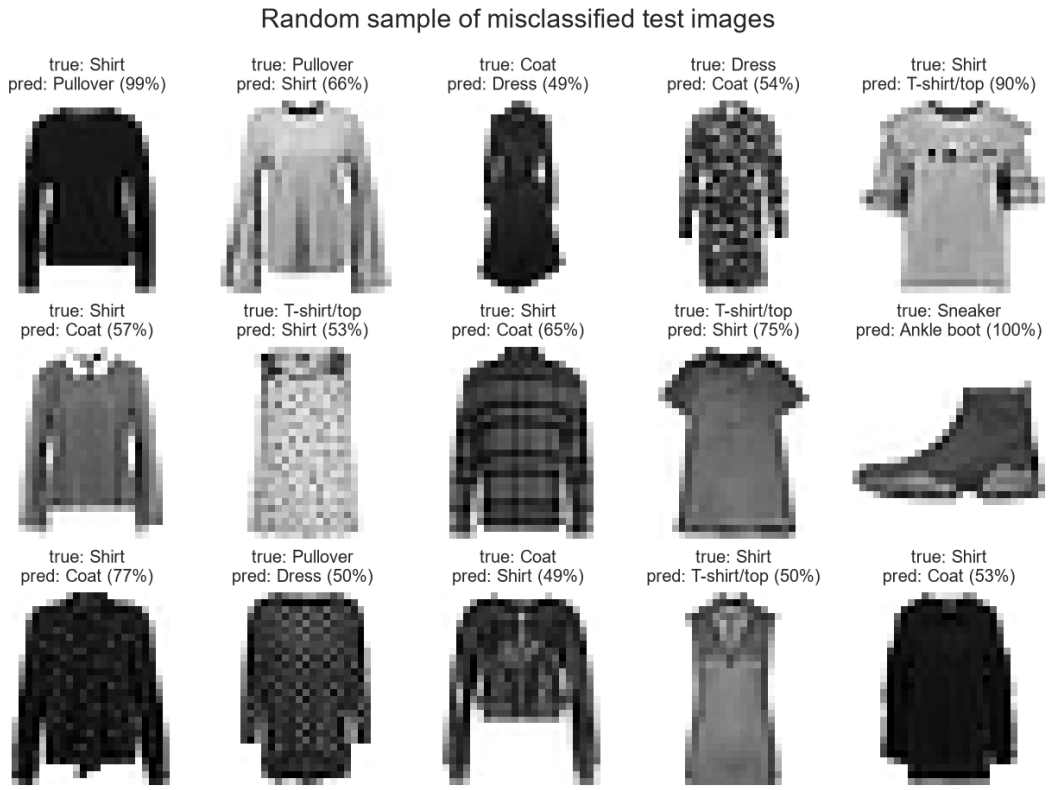


Figure 7. Random sample of misclassified test images, with the model's prediction and confidence.

6. Key Findings and Business Insights

- **Convolution matters.** Moving from a fully connected network to a CNN was the single biggest gain, because CNNs exploit the spatial structure of images.
- **Regularization turns accuracy into reliability.** The basic CNN began memorizing its training data. Augmentation, dropout and batch normalization cut the overfitting gap from 3.91 pts to 1.61 pts and gave the best validation and test results.
- **Errors are predictable.** The categories the exploration flagged as visually similar (T-shirt/top, Pullover, Coat, Shirt) account for most mistakes. Bag, Trouser, Sandal are recognized almost perfectly.
- **Confidence is a useful control knob. 83% of all test errors** had confidence below 0.9, so the confidence score separates reliable predictions from risky ones. The business can choose its own trade-off between automation and accuracy (table below and Figure 8). A few errors are confidently wrong, so periodic human spot checks are still worthwhile.

Confidence threshold	Images auto-tagged	Accuracy on auto-tagged	Sent to human (of 10,000)
none (tag everything)	100.0%	94.10%	0
0.50	98.9%	94.65%	107
0.70	93.4%	96.86%	657
0.90	84.5%	98.80%	1,551
0.95	79.7%	99.25%	2,032
0.98	73.5%	99.59%	2,653
0.99	69.1%	99.70%	3,090

Suggested starting point: a threshold of **0.95** auto-tags 79.7% of new images with 99.25% accuracy. A looser threshold of 0.9 automates 84.5% at 98.80% accuracy. The right setting depends on the relative cost of a wrong tag versus a manual review.

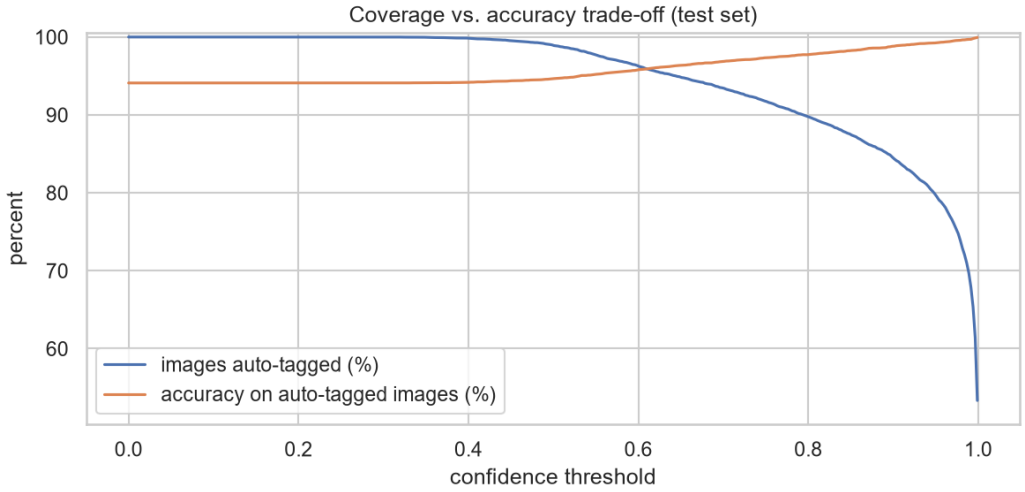


Figure 8. Raising the confidence threshold sends more images to people but makes the automated tags more accurate.

7. Model Flaws and Next Steps

Limitations

- **Benchmark data, not real photos.** Fashion-MNIST images are small, grayscale and perfectly centered. Real catalog photos are high-resolution, in color, and vary in background and pose, so accuracy must be re-validated on the retailer's own images.
- **Upper-body garment confusion.** Shirt remains weak (recall 0.81). At this resolution, details like collars and buttons are only a few pixels wide.
- **Limited tuning.** Each architecture was trained once with sensible defaults, without a systematic hyperparameter search or repeated runs with different random seeds. The recommended model's best epoch was 39 of the 40-epoch budget, so longer training would likely add a little more accuracy.

- **Confidence calibration not verified.** We used raw softmax confidence. Without calibration it can be over-confident on unusual images.

Recommended next steps

- **Transfer learning on real product images:** fine-tune a pretrained network (for example EfficientNet or ResNet) on the retailer's labeled color photos. This is the most likely route to production-grade accuracy.
- **Targeted data for hard classes:** collect more Shirt, T-shirt/top, Pullover and Coat examples, or add attribute labels such as sleeve length and collar type.
- **Hyperparameter optimization:** tune filter counts, dropout rates and learning rate (for example with KerasTuner), and average results over several seeds.
- **Calibration and monitoring:** apply temperature scaling so confidence scores match real accuracy. Then track the automation rate and reviewer corrections in production, and feed corrected labels back into regular retraining.

Appendix: Reproducibility

- **Notebook:** fashion_mnist_deep_learning.ipynb contains every step in this report, from data loading to the threshold analysis.
- **Environment:** Python 3.12, Keras 3.15.1 (backend: torch), scikit-learn, pandas, matplotlib and seaborn. Trained on a single GPU. Images use the channels-first layout native to PyTorch, which made training about 12 times faster than channels-last on this hardware.
- **Reproducibility:** random seed 42 for data splitting and weight initialization. GPU kernels are not fully deterministic, so re-runs can differ slightly in the last decimal places.
- **Saved artifacts:** results/final_model.keras (trained model), results/metrics.json (all reported metrics), figures/ (all charts).